



10

CONWAY'S GAME OF LIFE ON A TORUS



In [Chapter 3](#), we implemented Conway's Game of Life using Python and the `matplotlib` library. You may recall one interesting aspect of that project: it used toroidal boundary conditions. [Figure 3-2](#) on [page 48](#) showed how we were effectively treating the flat, 2D grid of the simulation as a 3D toroidal surface, thanks to how the boundary conditions stitched the edges together. In the [previous chapter](#), you were introduced to OpenGL and learned how to render 3D objects. Let's now put together your experience with the Game of Life and OpenGL to re-create the Conway's simulation in 3D, on an actual torus.

In this project, you'll start by computing the 3D geometry of a torus. Then you'll arrange the vertices of the torus in a way that's easy to draw and color in OpenGL. You'll set up a revolving camera to view the torus from all sides and also implement some basic lighting in the shaders. Finally, you'll adapt the Game of Life code from [Chapter 3](#) to color the grid on the torus. As the simulation proceeds, you'll see the Game of Life come alive on the surface of the torus!

Here are the main topics covered in this project:

- Using matrix math to construct the 3D geometry of a torus
- Implementing a coloring scheme for the Game of Life grid on the torus
- Implementing a revolving camera in OpenGL
- Implementing simple lighting in OpenGL

How It Works

Before getting into the code, let's consider how you can render, light, and view a 3D torus using OpenGL. It all begins with computing the vertices that make up the torus.

Computing Vertices

A torus is essentially a collection of circles, or rings, arranged in a circle around a central point. However, you can't actually draw circles with OpenGL; they need to be *discretized*, or represented as a series of vertices connected by straight lines. The simplified model in **Figure 10-1** illustrates how you can begin defining the torus as a collection of vertices.

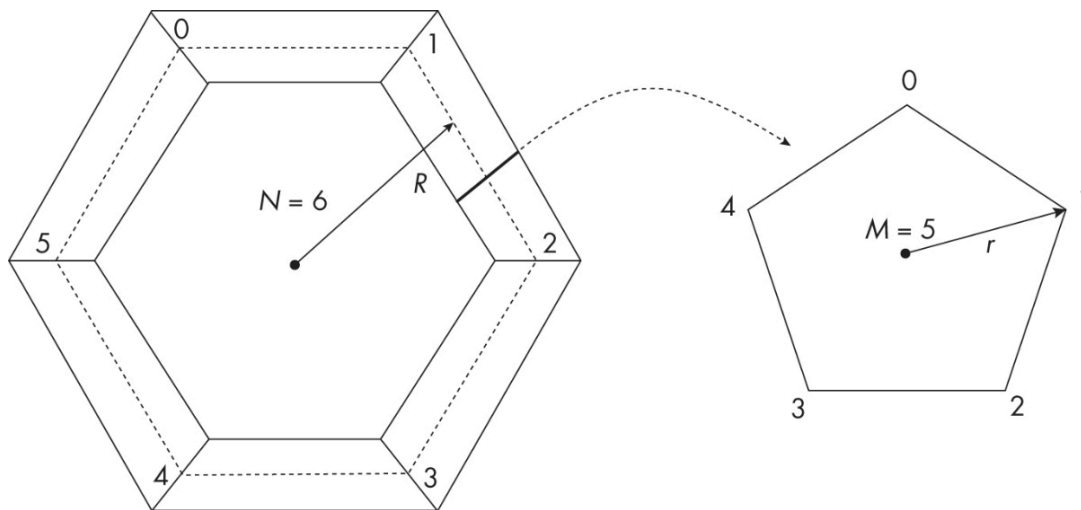


Figure 10-1: A torus rendering model. The simplified torus is on the left. An individual “ring” that makes up the torus is on the right.

The right side of [Figure 10-1](#) shows a ring with radius r discretized to have $M = 5$ points. The left side of [Figure 10-1](#) models a simple torus with major radius R by arranging $N = 6$ such discretized rings (labeled 0 through 5) around a central point. (The *major radius* is the distance from the center of the torus's hole to the center of its outer ring.) Don't worry about the blocky nature of the torus shown in the figure. It gets smoother as you increase the values of N and M .

The torus is filled out by drawing bands to connect adjacent rings. You'll draw the bands using `GL_TRIANGLE_STRIP` primitives, and each cell in the Game of Life simulation will consist of two adjacent triangles on a strip, which together form a *quad*, or quadrilateral. When a cell is ON you'll color its quad black, and when a cell is OFF you'll color it white.

To compute the vertices of the torus, you first need to define a coordinate system for it. Let's assume that the torus is on the XY plane, centered at the origin, with the line through the center of the torus aligned along the z-axis, as shown in [Figure 10-2](#).

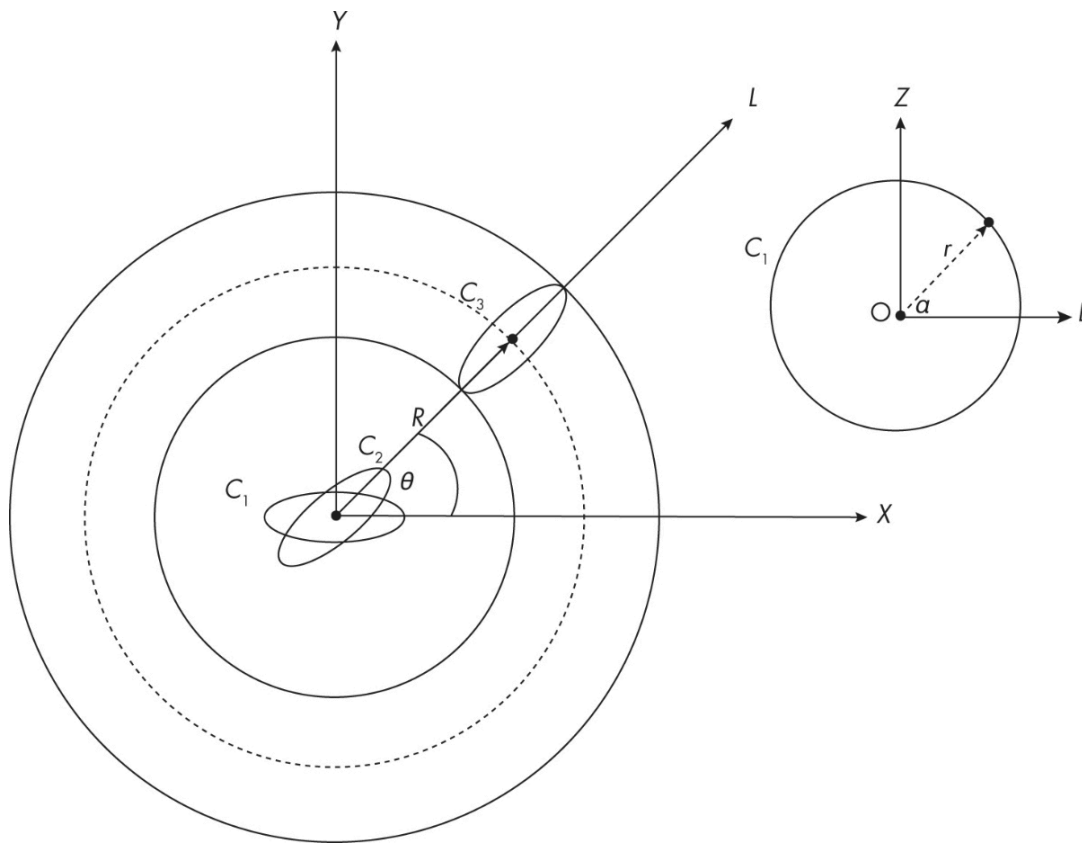


Figure 10-2: The rendering strategy for the torus

The vertices for a circle C_3 on the torus at an angle θ relative to the x-axis can be computed as follows:

1. Compute the vertices for a circle C_1 of radius r in the XZ plane and with the center at the origin.
2. Rotate the circle C_1 around the z-axis by the angle θ . This gives you the circle C_2 .
3. Translate the circle C_2 by R amount along the angle θ to get the circle C_3 in the correct position on the torus.

You may recall the use of parametric equations to define a circle in the spirograph project in [Chapter 2](#). We're going to use the same concept here. The vertices marking the circumference of a circle C_1 of radius r on the XZ plane centered at the origin are given by:

$$P = (r \cos(\alpha), 0, r \sin(\alpha))$$

Here, α is the angle made by the point P with respect to the x-axis. As α varies from 0 to 360 degrees (or 2π radians), a circle will be formed by the P points. Notice that the y-coordinates of the points are zero in the previous equation. This is expected, since the circle is on the XZ plane.

You must now rotate the points around the z-axis by angle θ . The rotation matrix for this operation is given by:

$$R_{\theta,z} = \begin{pmatrix} \cos(\theta) & -\sin(\theta) & 0.0 & 0.0 \\ \sin(\theta) & \cos(\theta) & 0.0 & 0.0 \\ 0.0 & 0.0 & 1.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 1.0 \end{pmatrix}$$

Once you rotate the points, you need to translate them to the right place. This is done using the following translation matrix. (This format was discussed in [Chapter 9](#).)

$$T = \begin{pmatrix} 1.0 & 0.0 & 0.0 & R \cos(\theta) \\ 0.0 & 1.0 & 0.0 & R \sin(\theta) \\ 0.0 & 0.0 & 1.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 1.0 \end{pmatrix}$$

Thus, the transformed points on a ring on the torus are given by:

$$P' = T \times R_{\theta,z} \times P$$

which is the same as:

$$P' = \begin{pmatrix} 1.0 & 0.0 & 0.0 & R \cos(\theta) \\ 0.0 & 1.0 & 0.0 & R \sin(\theta) \\ 0.0 & 0.0 & 1.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 1.0 \end{pmatrix} \cdot \begin{pmatrix} 1.0 & 0.0 & 0.0 & R \cos(\theta) \\ 0.0 & 1.0 & 0.0 & R \sin(\theta) \\ 0.0 & 0.0 & 1.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 1.0 \end{pmatrix} \cdot \begin{pmatrix} P_x \\ P_y \\ P_z \\ 1.0 \end{pmatrix}$$

In the previous equation, P is first multiplied by the rotation matrix, which aligns it correctly, and is then multiplied by the translation matrix, which “pushes” the points to the right place on the torus. Notice that P is represented with *homogeneous* coordinates $(x, y, z, 1.0)$, discussed in the [previous chapter](#).

Computing Normals for Lighting

For the torus to look nice, you’ll need to apply lighting to the geometry, which in turn means you’ll need to compute the *normal vectors* for the points P calculated in the previous section. The lighting on a surface depends on the orientation of the surface to the incoming light, and the orientation can be quantified by the normal vector, which is the vector perpendicular to the surface at a particular point. Take a look at [Figure 10-3](#) to see an example.

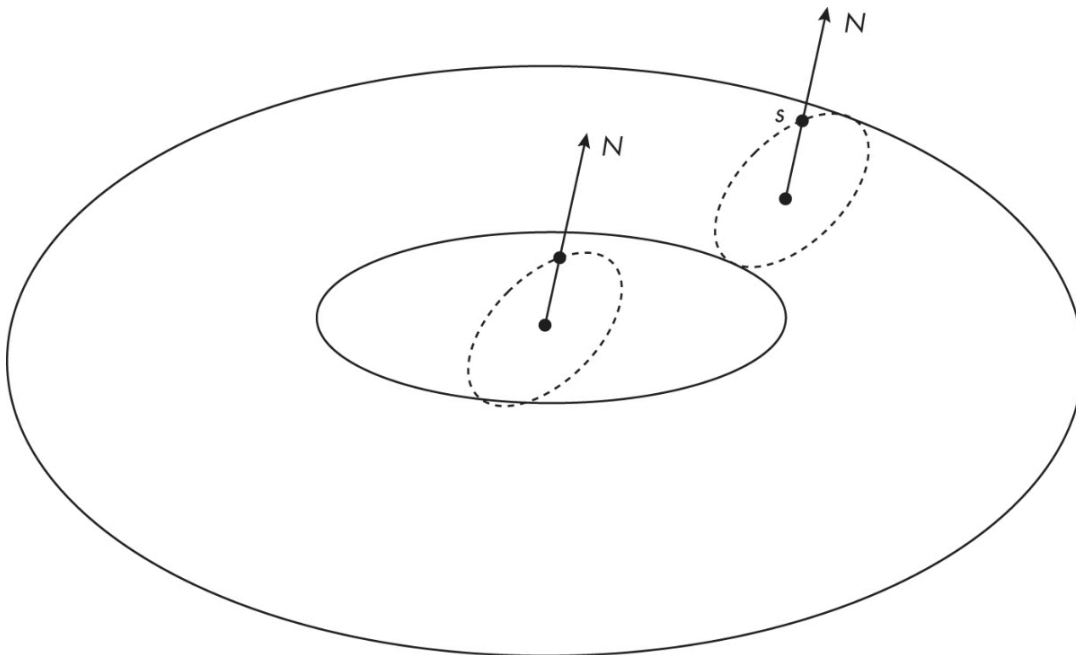


Figure 10-3: Computing normal vectors

Because of the geometry of the torus, the normal vector for point s on a ring is in the same direction as the line connecting point s to the center of the ring. This means that the normal vectors are the same as rotated points. The translation doesn't matter, as the direction of the normals is unaffected by it. You can therefore compute the normals as follows:

$$N = R_{\theta, Z} \times P$$

Note that you need to *normalize* the normal vectors before doing any lighting computations. You do this by dividing the normal by its magnitude.

The actual lighting will come from a single light source at a fixed position. This will be defined in the vertex shader.

Rendering

Now that you have the vertices and normals for the torus, let's talk about how to render it using OpenGL. You first need to split it up into bands, as shown in **Figure 10-4**. Each band is the region between two adjacent rings on the torus.

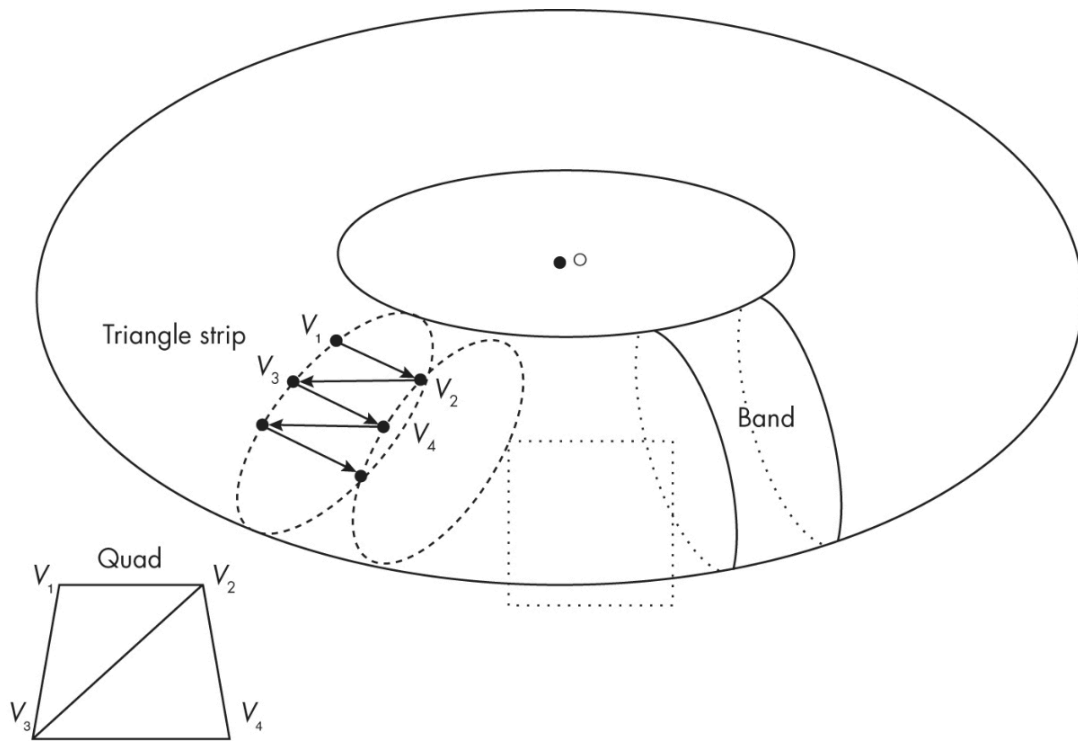


Figure 10-4: Rendering the torus with triangle strips

Each of the bands is rendered using OpenGL as a `GL_TRIANGLE_STRIP` primitive. Besides forming the building blocks of the torus, these triangle strips also provide a convenient way to create the Game of Life simulation grid: each cell in the grid is represented by a quad formed from two adjacent triangles in the strip. [Figure 10-5](#) gives a closer look at one band in the torus.

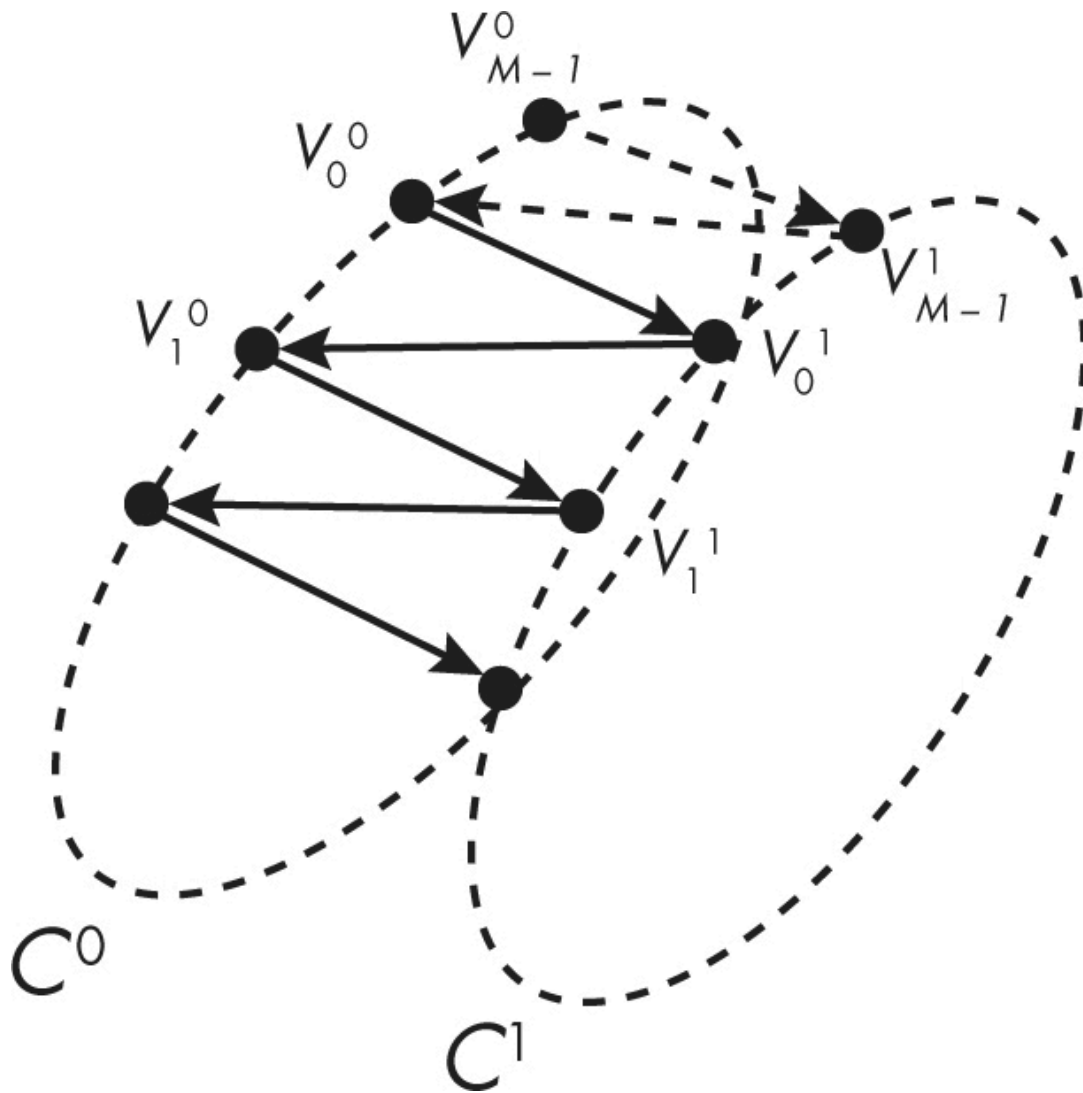


Figure 10-5: Rendering a band from triangle strips

The band is formed by the adjacent rings C^0 and C^1 . Each ring has M vertices. The triangle strip making up the band is formed as M pairs of vertices that zigzag back and forth between the rings:

$$V_0^0 \rightarrow V_0^1 \rightarrow V_1^0 \rightarrow V_1^1 \dots, V_{M-1}^0 \rightarrow V_{M-1}^1$$

But there's one additional pair of vertices you need to add:

$V_0^0 \rightarrow V_0^1$. You need to repeat the first two vertices to close out the gap at the end of the band. Hence, the total number of vertices in the triangle strip that forms the band is given by $2 \times M + 2$.

The band shown in [Figure 10-5](#) was composed of rings $C^0 \rightarrow C^0$.

The torus is divided into N bands, where N is the number of rings:

$$(C^0 \rightarrow C^1 \rightarrow C^1 \rightarrow C^2 \dots, C^{N-1} \rightarrow C^0)$$

Notice how the last band wraps around by coming back to the first ring, C^0 . This means the total number of vertices needed to render the torus is given by $N \times (2 \times M + 2)$. You'll see more details of the implementation as you look at the code.

Now let's take a look at the coloring scheme for the torus.

Coloring the Triangle Strips

You need to color the cells in the Game of Life simulation individually.

You know that each cell is a quad—two triangles that are part of a larger triangle strip. As an example, the quad made of vertices

$V_0^0 \rightarrow V_0^1 \rightarrow V_1^0 \rightarrow V_1^1$ is composed of two triangles:
 $V_0^0 \rightarrow V_0^1 \rightarrow V_1^0$ and $V_1^1 \rightarrow V_1^0 \rightarrow V_0^1$. Each vertex has a cor-

responding color associated with it, which is a triplet of the form (r, g, b) , denoting the red, green, and blue components of the color. By default, the color of the first vertex in the quad (in this case, V_0^0) sets the value for the first triangle in the quad, and the color of the second vertex (V_0^1) sets the value for the second triangle in the quad. As long as you set these two colors to be identical, you'll color the quad uniformly. We'll discuss OpenGL's vertex color convention further when we look at the code.

NOTE The OpenGL function called `glProvokingVertex()` changes the convention of which color value is mapped to the vertex.

Controlling the Camera

To view the torus, you'll create a camera that revolves around the origin of the 3D scene and looks down at an angle from above. **Figure 10-6** shows the camera setup.

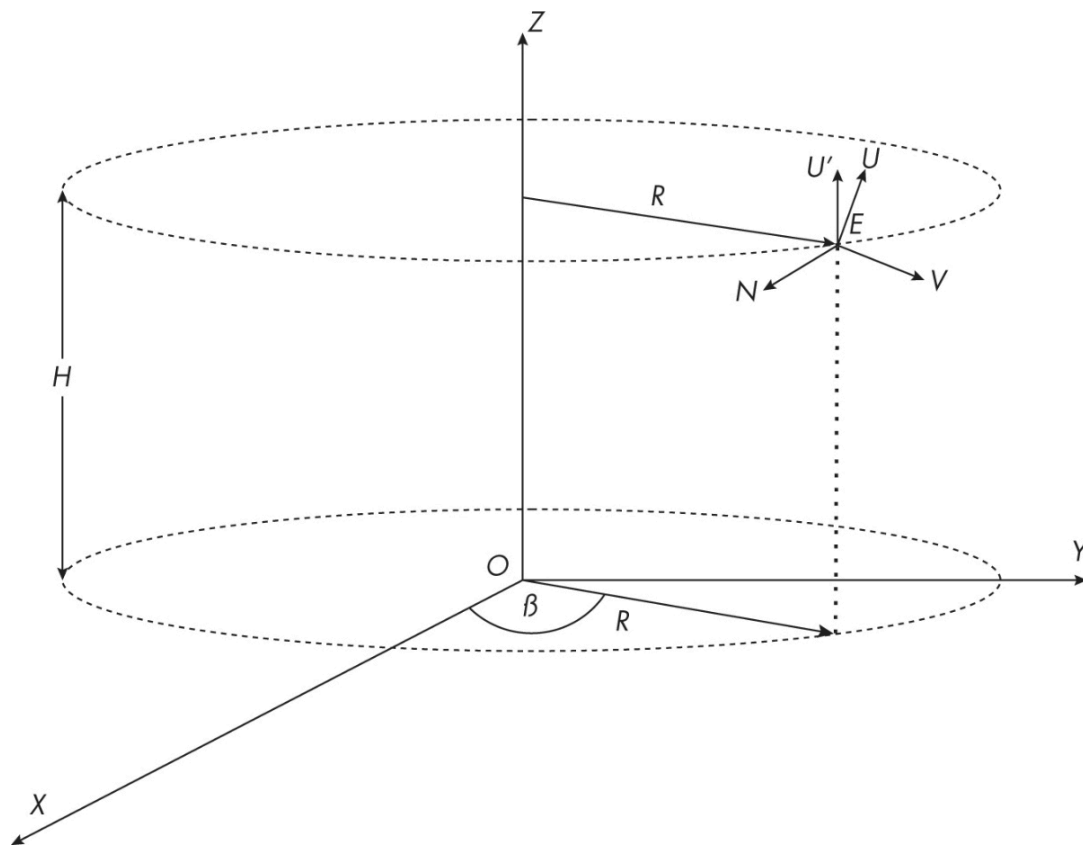


Figure 10-6: Implementing a revolving camera

Think of the camera, represented by point E , as being placed on a cylinder of radius R at a height H from the origin. The camera points toward the origin O . The camera is defined by the mutually perpendicular vectors V , U , and N , where V is the *view vector* that points from E to O ; U is the *up vector*, which is up relative to the camera; and N is a vector perpendicular to both V and U . For every time step, you move the camera by a constant distance along the rim of the cylinder. This movement is parameterized by the angle β , as shown in **Figure 10-6**. As you learned in **Chapter 9**, you use a `lookAt()` method to set up the view, which takes three parameters: the eye, the center, and the up vector. The center is just the origin: $(0, 0, 0)$. The 3D coordinates of the eye are given by:

$$E = (R \cos(\beta), R \sin(\beta), H)$$

As the camera moves along the rim of the cylinder, it will keep its direction pointed to O , and the up vector U will keep changing as well. To compute the up vector U , start with an initial guess U' that's parallel to the z-axis. Then find N , the vector perpendicular to the plane given by U' and V . It can be calculated as:

$$N = V \times U'$$

N is given by the cross product of V and U' . Now, what happens if you take the cross product of N and V ? You get a vector perpendicular to the NV plane, which is the up vector U that you're looking for!

$$U = N \times V = (V \times U') \times V$$

Once U is calculated, make sure that it's normalized before use. Once that's done, you'll have everything you need to set up the camera with `lookAt()` : E (the eye), O (the center), and U (the up vector).

Mapping the Grid to the Torus

Finally, let's look at how the 2D Game of Life simulation grid maps to the 3D torus because of the grid's toroidal boundary conditions.

Figure 10-7 shows the mapping.

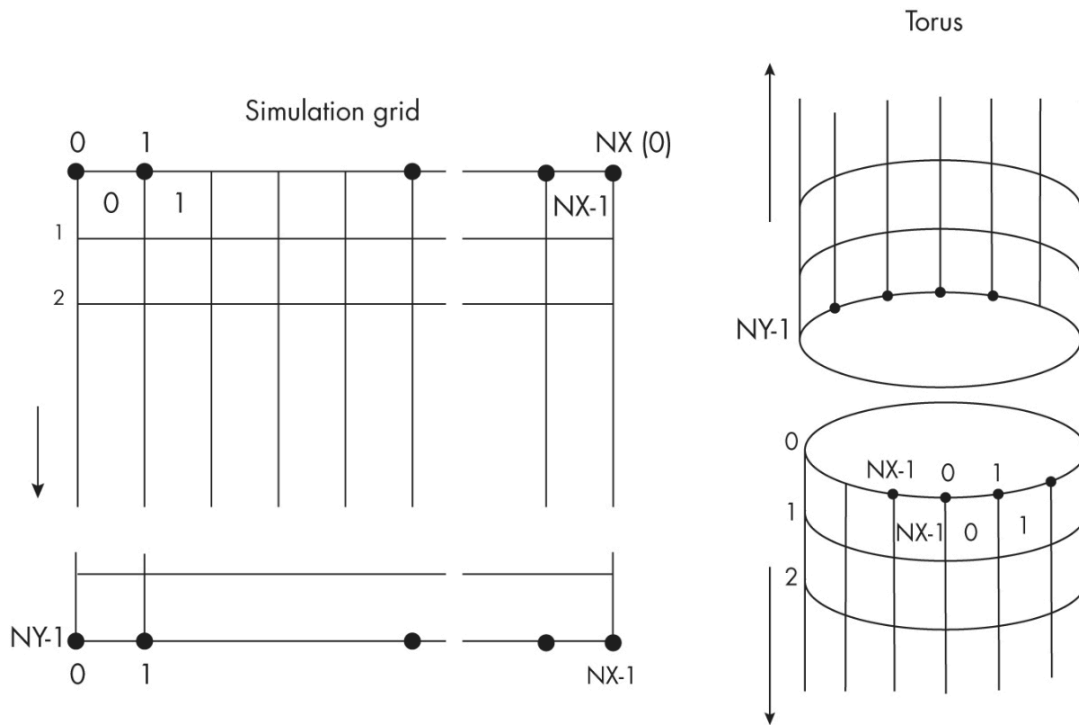


Figure 10-7: Mapping the simulation grid to a torus

The two-dimensional Game of Life grid has NX columns and NY rows. You can see on the right of the figure how the NX points that divide each row wrap around the tube of the torus. The point indices go from 0 to $NX - 1$. The next index, NX , is the same as 0 because of the wrap. A similar wrap happens in the y-direction, where you have NY cells: the point with index NY is the same as the point with index 0.

You saw earlier how each ring in the torus is discretized to have M points. To go from the two-dimensional grid to the torus, set $NX = M$. Similarly, set $NY = N$, where N is the number of bands on the torus.

Requirements

We'll use `PyOpenGL` and `GLFW` for OpenGL rendering, as in [Chapter 9](#), and `numpy` for matrix/vector computations.

The Code

The code for this project is organized into several files:

torus.py This contains the geometry computation and rendering code for the torus.

gol.py This implements Conway's Game of Life, adapted from [**Chapter 3**](#).

camera.py This contains the implementation for the revolving camera for viewing the torus.

gol_torus.py This is the main file that sets up OpenGL and GLFW and calls the rendering code from other modules.

The full project code is available online at [**https://github.com/mkvenkit/pp2e/blob/main/gol_torus**](https://github.com/mkvenkit/pp2e/blob/main/gol_torus).

Rendering the Torus

We'll first consider the code for rendering the torus, which is encapsulated in a class called `Torus` defined in the file *torus.py*. To see the complete code listing, skip ahead to [**"The Complete Torus Rendering Code"**](#) on [**page 203**](#).

Defining the Shaders

First, define the GLSL shaders for the torus. Here's the vertex shader, which takes the per-vertex attributes of position, color, and normal and computes the transformed inputs for the fragment shader:

```
strVS = """
```

```
# version 410 core
```

```
layout(location = 0) in vec3 aVert;
```

```
layout(location = 1) in vec3 aColor;
```

```

layout(location = 2) in vec3 aNormal;

uniform mat4 uMVMMatrix;

uniform mat4 uPMatrix;

❶ flat out vec3 vColor;

❷ out vec3 vNormal;

❸ out vec3 fragPos;

void main() {

    // transform vertex

    gl_Position = uPMatrix * uMVMMatrix * vec4(aVert, 1.0);

    ❹ fragPos = aVert;

    vColor = aColor;

    vNormal = aNormal;

}

"""

```

You define the vertex shader code as a string stored in `strVS`. The attribute variables for the shader are `aVert`, `aColor`, and `aNormal`, representing the coordinates, color, and normal vector for each vertex. Notice the `flat` qualifier when you initialize `vColor`, one of the outputs of the vertex shader ❶, indicating that this variable won't be interpolated in the fragment shader. In effect, we're saying that this variable will remain constant across the primitive (one of the triangles in a triangle strip). This ensures that every Game of Life cell will be a single color. This type of shading across a primitive is called *flat shading*.

ing. The next vertex shader output is `vNormal` ❷, which by default will be interpolated in the fragment shader. You need this so you can compute lighting across a primitive, but later you'll see how to modify this shader code to support flat shading. The other output is called `fragPos` ❸. In the main shader code, you set this output to `aVert` ❹ so it can be passed into the fragment shader for lighting calculations. The shader also computes `gl_Position` and passes to the fragment shader the color and normal data as it is received.

Here's the fragment shader, which applies lighting and computes the final color of the fragment. It's defined as another string, called `strFS`.

```
strFS = ""

# version 410 core

flat in vec3 vColor;

in vec3 vNormal;

in vec3 fragPos;

out vec4 fragColor;

void main() {

    ❶ vec3 lightPos = vec3(10.0, 10.0, 10.0);

    ❷ vec3 lightColor = vec3(1.0, 1.0, 1.0);

    ❸ vec3 lightDir = normalize(lightPos - fragPos);

    float diff = max(dot(vNormal, lightDir), 0.0);

    vec3 diffuse = diff * lightColor;
```

```

float ambient = 0.1;

④ vec3 result = (ambient + diffuse) * vColor.xyz;

fragColor = vec4(result, 1.0);

}

"""

```

Notice that the color, normal, and fragment position variables, which were outputs from the vertex shader, are now inputs to the fragment shader. In the main shader code, you define the position ❶ and color ❷ for a light source. Then you compute the light direction ❸. The resulting color ❹ is a mix of ambient and diffuse components of the light and is set as the output from the fragment shader.

Keep in mind that `fragPos` and `vNormal` are computed for each fragment via interpolation, whereas `vColor` is constant for a given primitive. The net effect is that the intrinsic color of a primitive (a triangle strip, in this case) remains constant, while the perceived color varies across the primitive based on its orientation with respect to the light source. This is exactly what you need to set each Game of Life cell to a solid color, while varying that color to create the appearance of lighting.

Initializing the Torus Class

Now let's look at the initialization code in the `Torus` class constructor:

```

class Torus:

    """ OpenGL 3D scene class """

    # initialization

```

```
❶ def __init__(self, R, r, NX, NY):

    global strVS, strFS

    # modify shader for flat shading

    # create shader

❷ self.program = glutils.loadShaders(strVS, strFS)

    glProvokingVertex(GL_FIRST_VERTEX_CONVENTION)

    self.pMatrixUniform = glGetUniformLocation(self.program,

                                                b'uPMatrix')

    self.mvMatrixUniform = glGetUniformLocation(self.program,

                                                b'uMVMatrix')

    # torus geometry

    self.R = R

    self.r = r

    # grid size

    self.NX = NX

    self.NY = NY

    # no. of points

❸ self.N = self.NX

    self.M = self.NY
```

```
# time

self.t = 0

# compute parameters for glMultiDrawArrays

M1 = 2*self.M + 2

❹ self.first_indices = [2*M1*i for i in range(self.N)]

self.counts = [2*M1 for i in range(self.N)]

# colors: {(i, j) : (r, g, b)}

# with NX * NY entries

❺ self.colors_dict = self.init_colors(self.NX, self.NY)

# create an empty array to hold colors

❻ self.colors = np.zeros((3*self.N*(2*self.M + 2), ), np.float32)

# get vertices, normals, indices

❼ vertices, normals = self.compute_vertices()

❽ self.compute_colors()

# set up vertex buffer objects

❾ self.setup_vao(vertices, normals, self.colors)
```

The `Torus` class has the following parameters in its constructor ❶: the radius `R` of the outer ring of the torus, the radius `r` of the torus tube, and `NX` and `NY` for the number of Game of Life simulation cells in the x- and y-directions. The constructor's first order of business is to load the shaders. You use the `loadShaders()` method ❷ defined in the

common *glutils.py* file. In the subsequent lines, you store the variables passed into the `Torus` constructor in instance variables such as `self.R` so they can be accessed later from other methods. Then you set `N`, the number of points on the outer circle of the torus, to be `NX`, the number of cells in the x-direction ❸. You similarly set `M`, the number of points along the smaller radius `r` of the torus, to `NY`. This scheme is discussed in the [“Mapping the Grid to the Torus”](#) section.

Next, you do some additional preparation for rendering the triangle strips that will form the bands along the outer ring of the torus. You'll eventually be using the `glMultiDrawArrays()` OpenGL method to render all the triangle strips at once. This method is an efficient way to draw multiple triangle strip primitives using just one function call. As you saw in the [“Rendering”](#) section, each triangle strip has $2M + 2$ vertices, and you have N of these strips. So, the starting indices for these triangle strips will be $[0, (2M + 2), (2M + 2) \times 2, \dots, (2M + 2) \times N]$. Accordingly, you set `first_indices` and `counts` ❹, which will be required parameters when you call `glMultiDrawArrays()`.

The `init_colors()` method ❺ initializes `color_dict`, which maps each grid cell to a color—black or white. We'll look at the details of the `init_colors()` method soon. You initialize the `numpy` array `colors` to zeros ❻. You'll later populate this array with the correct values. You conclude the constructor by computing the vertices and normals for the torus ❼, as well as the colors ❽, and by setting up the vertex array object (VAO) for rendering the torus ❾.

Now let's look at the `Torus` class's `init_colors()` method that was just mentioned:

```
def init_colors(self, NX, NY):

    """initialize color dictionary"""

    colors = {}
```

```
c1 = [1.0, 1.0, 1.0]

for i in range(NX):

    for j in range(NY):

        ❶ colors[(i, j)] = c1

return colors
```

The `init_colors()` method creates a dictionary called `colors` mapping from simulation cell index (i, j) to the color that should be applied to the cell. To begin, you simply set all the cell color values to `c1`, which is just plain white ❶. As the Game of Life simulation unfolds, the values in this dictionary will be updated to turn cells on and off.

Calculating the Vertices

The next few methods we'll consider work together to compute all the torus vertices. We begin with the `compute_vertices()` method:

```
def compute_vertices(self):

    R, r, N, M = self.R, self.r, self.N, self.M

    # create an empty array to hold vertices/normals

    vertices = []

    normals = []

    for i in range(N):

        # for all M points around a ring

        for j in range(M+1):
```

```
# compute angle theta of point
```

```
❶ theta = (j % M) * 2 * math.pi / M
```

```
#---ring #1-----
```

```
# compute angle
```

```
❷ alpha1 = i * 2 * math.pi / N
```

```
# compute transforms
```

```
❸ RM1, TM1 = self.compute_rt(R, alpha1)
```

```
# compute points
```

```
❹ Pt1, NV1 = self.compute_pt(r, theta, RM1, TM1)
```

```
#---ring #2-----
```

```
# index of next ring
```

```
❺ ip1 = (i + 1) % N
```

```
# compute angle
```

```
❻ alpha2 = ip1 * 2 * math.pi / N
```

```
# compute transforms
```

```
RM2, TM2 = self.compute_rt(R, alpha2)
```

```
# compute points
```

```
Pt2, NV2 = self.compute_pt(r, theta, RM2, TM2)
```

```
# store vertices/normals in right order for
GL_TRIANGLE_STRIP
```

```
⑦ vertices.append(Pt1[0:3])

vertices.append(Pt2[0:3])

# add normals

normals.append(NV1[0:3])

normals.append(NV2[0:3])

# return vertices and colors in correct format

⑧ vertices = np.array(vertices, np.float32).reshape(-1)

normals = np.array(normals, np.float32).reshape(-1)

# print(vertices.shape)

return vertices, normals
```

The `compute_vertices()` method begins by creating empty lists to store the vertices and normals. Then you compute the vertices and normals for the torus, using a nested loop to implement the strategy we discussed in the **“Rendering”** section. The outer loop iterates over the `N` rings that make up the torus. The inner loop iterates over the `M` points on each of those rings. Within the loops, you first compute the angle `theta` subtended by a point on the ring with index `j` ①. You use `j % M` and have the inner loop iterate over range `[0, M+1)` so that when `j` is equal to `M`, `(j % M)` rolls back to `0`. This is to complete the last segment of the ring.

The torus is rendered as a set of bands (triangle strips), and each band consists of two adjacent rings of points. You compute `alpha1`, the an-

gle subtended by the first ring in a band, at index `i` ❷, and use `alpha1` to compute the rotation and translation matrices for this first ring using the `compute_rt()` method ❸. Then you pass these matrices to the `compute_pt()` method to calculate the vertex and normal for the point on the ring at angle `theta` ❹. We'll look at how the `compute_rt()` and `compute_pt()` methods work soon.

Next, you move on to the adjacent ring at index `i+1`, using `ip1 = (i+1) % N` to ensure that you roll back to zero at the end ❺. You compute the angle `alpha2` of the ring at index `ip1` ❻ and then compute the vertex and normal for the point on the `ip1` ring at angle `theta`, just as you did for the first ring.

Beginning at ❼, you append the vertices and normals for the adjacent rings to the lists you created at the beginning of the method. You pick only the first three coordinates of each vertex and normal, as in `Pt1[0:3]`, since all the matrix transformations are done with homogeneous coordinates in the form (x, y, z, w) , and you need only (x, y, z) . This action stores the vertices and normals in a Python list of triplets of the form `[[x1, y1, z1], [x2, y2, z2], ...]`. However, OpenGL expects vertex attributes to be provided in a flat array with a known size. Therefore, you convert the `vertices` and `normals` lists into `numpy` arrays of 32-bit floats ❽, using `reshape(-1)` to ensure that they're flat arrays of the form `[x1, y1, z1, x2, y2, z2, ...]`.

Now let's look at the `compute_rt()` and `compute_pt()` methods that helped you compute the vertices and normals. We'll start with `compute_rt()`, which calculates the rotation and translation matrices needed to render a given ring in the torus:

```
def compute_rt(self, R, alpha):
```

```
    # compute position of ring
```

```
    ❶ Tx = R*math.cos(alpha)
```

```
Ty = R*math.sin(alpha)
```

```
Tz = 0.0
```

```
# rotation matrix
```

```
❷ RM = np.array([
```

```
    [math.cos(alpha), -math.sin(alpha), 0.0, 0.0],
```

```
    [math.sin(alpha), math.cos(alpha), 0.0, 0.0],
```

```
    [0.0, 0.0, 1.0, 0.0],
```

```
    [0.0, 0.0, 0.0, 1.0]
```

```
], dtype=np.float32)
```

```
# translation matrix
```

```
❸ TM = np.array([
```

```
    [1.0, 0.0, 0.0, Tx],
```

```
    [0.0, 1.0, 0.0, Ty],
```

```
    [0.0, 0.0, 1.0, Tz],
```

```
    [0.0, 0.0, 0.0, 1.0]
```

```
], dtype=np.float32)
```

```
return (RM, TM)
```

You first calculate the translation components of the matrix ❶, using parametric equations. Then you create the rotation matrix ❷ and translation matrix ❸ as `numpy` arrays. You've seen these matrices be-

fore in the **“Computing Vertices”** section. You return the arrays at the end of the method.

Here’s the other helper method, `compute_pt()`, which uses the translation and rotation matrices to determine the vertex and normal vector of a given point on a ring of the torus:

```
def compute_pt(self, r, theta, RM, TM):

    # compute point coords

    ❶ P = np.array([r*math.cos(theta), 0.0, r*math.sin(theta), 1.0],

                   dtype=np.float32)

    # print(P)

    # apply rotation - this also gives us the vertex normals

    ❷ NV = np.dot(RM, P)

    # apply translation

    ❸ Pt = np.dot(TM, NV)

    return (Pt, NV)
```

You compute the point `P` at angle `theta` on a ring lying on the XZ plane ❶. Then you apply a rotation to this point by multiplying it by the rotation matrix ❷. This also gives you the point’s normal vector. You multiply the normal by the translation matrix to give you the vertex on the torus ❸.

Managing Cell Colors

Now we'll examine some methods that help set the colors of the cells on the torus. First is the `compute_colors()` method, which we originally called as part of the `Torus` class's constructor. It sets the color of each triangle in the triangle strips that make up the torus, based on the values determined by the Game of Life simulation.

```
def compute_colors(self):

    R, r, N, M = self.R, self.r, self.N, self.M

    # the points on the ring are generated on the X-Z plane

    # then they are rotated and translated into the correct

    # position on the torus

    # for all N rings around the torus

    for i in range(N):

        # for all M points around a ring

        for j in range(M+1):

            # j value

            jj = j % M

            # store colors - same color applies to (V_i_j, V_ip1_j)

            ❶ col = self.colors_dict[(i, jj)]

            # get index into array

            ❷ index = 3*(2*i*(M+1) + 2*j)

            # set color
```

③ `self.colors[index:index+3] = col`

④ `self.colors[index+3:index+6] = col`

This method follows the logic described in [“Coloring the Triangle Strips”](#) on [page 185](#) to update the values in the `colors` array, which was initialized as an array of all zeros. You retrieve the color for cell `(i, jj)` from `colors_dict`, the dictionary mapping cells to colors that you created earlier ①. (You define `jj = j % M` so it rolls over to zero at the end.) Then you compute the index into the `colors` array at which you should update the new computed values ②. Each pair of rings that makes up a band has `2*(M+1)` vertices, and there are `N` of these pairs. Starting at each location in the array, you store three sequential values (the RGB components of a cell’s color). Hence, the index of the *j*th color in a ring for the *i*th segment of the torus will be given by `3*(2*i*(M+1) + 2*j)`. Note that you use `j` and not `jj` in computing the index, since you’re storing the computed values here and you don’t want the index to roll over to zero. Now that you have the index, you update the `colors` array with the new computed values. You update the array both at `[index:index+3]` ③ and at `[index+3:index+6]` ④ since each cell on the torus is a quad, made of two adjacent triangles.

Let’s now look at `recalc_colors()`, a method for updating the color values stored on the GPU at each step in the Game of Life simulation:

```
def recalc_colors(self):
```

```
    # get colors
```

```
    self.compute_colors()
```

```
    # bind VAO
```

```
    glBindVertexArray(self.vao)
```

```
glBindBuffer(GL_ARRAY_BUFFER, self.colorBuffer)

# set buffer data

❶ glBufferSubData(GL_ARRAY_BUFFER, 0, 4*len(self.colors),
self.colors)

# unbind VAO

glBindVertexArray(0)
```

For every step of the simulation, the colors of the cells are updated, which means you need to update the colors of all the triangle strips on the torus, and you need to do it efficiently so you don't slow down the rendering. The `recalc_colors()` method does this using the OpenGL `glBufferSubData()` method ❶. The vertices, normals, and colors are stored in attribute arrays on the GPU. The vertices and normals don't change, so you computed them only once in the beginning, with a call to `compute_vertices()` in the class's constructor method. When the colors change, `glBufferSubData()` updates the color attribute arrays rather than creating them afresh.

Drawing the Torus

Finally, here's the `render()` method that draws the torus:

```
def render(self, pMatrix, mvMatrix):

    # use shader

    ❶ glUseProgram(self.program)

    # set proj matrix

    ❷ glUniformMatrix4fv(self.pMatrixUniform, 1, GL_FALSE, pMatrix)
```

```
# set modelview matrix

❸ glUniformMatrix4fv(self.mvMatrixUniform, 1, GL_FALSE,
mvMatrix)

# bind VAO

❹ glBindVertexArray(self.vao)

# draw

❺ glMultiDrawArrays(GL_TRIANGLE_STRIP, self.first_indices,

                    self.counts, self.N)

# unbind VAO

glBindVertexArray(0)
```

This method is similar to the render methods you saw in the [previous chapter](#). You make a call to use the shader program ❶ and set the projection ❷ and modelview ❸ matrix uniform variables. Then you bind to the vertex array object ❹, which you created by calling `setup_vao()` in the class's constructor. The VAO has all the attribute array buffers you need. Next, you use the `glMultiDrawArrays()` method to draw `N` triangle strips ❺. You already computed `first_indices` and `counts` in the `Torus` constructor.

Implementing the Game of Life Simulation

In [Chapter 3](#), you implemented Conway's Game of Life (GOL) by using `matplotlib` to visualize the updated values of the simulation grid. Here you'll adapt the earlier implementation to update a dictionary of cell colors instead, which will be used to update the colors of the torus. The relevant code is encapsulated in a class called `GOL`, declared in

the file *gol.py*. To see the complete code listing, skip ahead to [“The Complete Game of Life Simulation Code”](#) on [page 209](#).

The Class Constructor

First, let's look at the `GOL` class constructor:

class `GOL`:

```
❶ def __init__(self, NX, NY, glider):  
  
    """GOL constructor"""  
  
    # a grid of NX x NY random values  
  
    self.NX, self.NY = NX, NY  
  
    if glider:  
  
        ❷ self.addGlider(1, 1, NX, NY)  
  
    else:  
  
        ❸ self.grid = np.random.choice([1, 0], NX * NY,  
  
                                     p=[0.2, 0.8]).reshape(NX, NY)
```

The `GOL` constructor takes as input the grid dimensions `NX` and `NY`, as well as a Boolean flag `glider` ❶. If this flag is set, you initialize the simulation grid with the “glider” pattern using the `addGlider()` method ❷. Since we already discussed this method in [Chapter 3](#), we won't examine it here. If the `glider` flag isn't set, you just initialize the grid to random ones and zeros ❸.

The `GOL` class uses an `update()` method to update the simulation grid at each time step. Again, this is identical to the previous implementation.

The `get_colors()` Method

The `get_colors()` method is what distinguishes this chapter's Game of Life implementation from that of [Chapter 3](#). The method builds up a dictionary mapping of each Game of Life cell to its color value at a given step in the simulation: black for ON or white for OFF. This dictionary is passed to the `Torus` object when the scene is updated.

```
def get_colors(self):

    colors = {}

    ❶ c1 = np.array([1.0, 1.0, 1.0], np.float32)

    ❷ c2 = np.array([0.0, 0.0, 0.0], np.float32)

    for i in range(self.NX):

        for j in range (self.NY):

            if self.grid[i, j] == 1:

                colors[(i, j)] = c2

            else :

                colors[(i, j)] = c1

    return colors
```

Here you iterate through all the cells in the simulation grid and set the RGB colors based on whether the grid value is `0` or `1`. The possible colors are defined as `c1` for white ❶ or `c2` for black ❷. These colors will be used while rendering the torus.

[Creating the Camera](#)

In [“Controlling the Camera”](#) on [page 185](#), we discussed how to build a camera that orbits around the torus. Now let’s look at the implementation. The code is encapsulated in the class `OrbitCamera`, which is declared in the file `camera.py`. To see the complete code listing, skip ahead to [“The Complete Camera Code”](#) on [page 211](#).

Constructing the Class

Here’s the constructor for the `OrbitCamera` class:

```
class OrbitCamera:
```

```
    """helper class for viewing"""
```

```
    def __init__(self, height, radius, beta_step=1):
```

```
        ❶ self.radius = radius
```

```
        ❷ self.beta = 0
```

```
        ❸ self.beta_step = beta_step
```

```
        ❹ self.height = height
```

```
        # initial eye vector is (-R, 0, -H)
```

```
        rr = radius/math.sqrt(2.0)
```

```
        ❺ self.eye = np.array([rr, rr, height], np.float32)
```

```
        # compute up vector
```

```
        ❻ self.up = self._compute_up_vector(self.eye )
```

```
        # center is origin
```

```
⑦ self.center = np.array([0, 0, 0], np.float32)
```

You start by setting the camera parameters passed into the `OrbitCamera` constructor. These include the camera's orbit radius ① and `beta`, the angle that the view vector (projected on the XY plane) makes with the x-axis ②. You also set the amount `beta` should increment with each time step of the camera rotation ③ and the height of the camera from the XY plane ④.

Next, you set the initial value of the eye position to be midway between the positive x- and positive y-axis, at a distance `R` from the origin, suspended at the specified `height` ⑤. You can calculate this as:

$$E = \left(\frac{R}{\sqrt{2}}, \frac{R}{\sqrt{2}}, height \right)$$

Finally, you compute the camera's up vector ⑥ and set the center as the origin (0, 0, 0) ⑦. Remember that these are the pieces of information, along with the eye position, that OpenGL needs to model a camera.

Calculating the Up Vector

Here's the method that you call in the `OrbitCamera` class constructor to compute the up vector:

```
def __compute_up_vector(self, E):
```

```
    # N = (E x k) x E
```

```
    Z = np.array([0, 0, 1], np.float32)
```

```
    ① U = np.cross(np.cross(E, Z), E)
```

```
# normalize
```

```
❷ U = U / np.linalg.norm(U)
```

```
return U
```

The `__compute_up_vector()` method calculates the up vector `U` based on the method we discussed earlier in [“Controlling the Camera”](#) on [page 185](#). Specifically, you use cross products and the initial up vector guess of (0, 0, 1) to compute the correct up vector ❶. Then you normalize the up vector ❷ before returning it.

Rotating the Camera

The `OrbitCamera` class's `rotate()` method is called every time you need to rotate the camera around the torus by one step. Here's the method's definition:

```
def rotate(self):
```

```
    """rotate by one step and compute new camera parameters"""
```

```
    ❶ self.beta = (self.beta + self.beta_step) % 360
```

```
    # recalculate eye E
```

```
    ❷ self.eye = np.array([self.radius*math.cos(math.radians(self.beta)),
```

```
                        self.radius*math.sin(math.radians(self.beta)),
```

```
                        self.height], np.float32)
```

```
    # up vector
```

```
    ❸ self.up = self.__compute_up_vector(self.eye)
```

You increase the angle `beta` by the increment `beta_step`, using the `%` operator to ensure that the angle rolls over to 0 when it reaches 360 degrees ❶. Then you use the new `beta` value to compute the updated eye position ❷, and you use the new eye position to compute the new up vector with the `__compute_up_vector()` method ❸.

Putting Everything Together

You've built all the classes necessary for rendering the torus. Now you need some code to bring those classes together, create and manage the OpenGL window, and coordinate the rendered objects. Create the class `RenderWindow` (defined in `gol_torus.py`) for this purpose. It's similar to the `RenderWindow` class used in [Chapter 9](#), so we'll discuss only the parts of the code that are unique to the current project. To see the complete code listing, skip ahead to [“The Complete RenderWindow Code”](#) on [page 211](#).

The main() Function

Before we examine the `RenderWindow` class, let's look the program's `main()` function, which sets the whole simulation in motion. This function is also defined in `gol_torus.py`.

```
def main():

    print("Starting GOL. Press ESC to quit.")

    # parse arguments

    parser = argparse.ArgumentParser(description="Runs Conway's
    Game of Life

                                simulation on a Torus.")

    # add arguments
```

```
❶ parser.add_argument('--glider', action='store_true', required=False)

args = parser.parse_args()

glider = False

if args.glider:

    ❷ glider = True

❸ rw = RenderWindow(glider)

❹ rw.run()
```

You add a command line argument called `--glider` so you can bring up the torus with just a glider pattern on it ❶ and set the corresponding flag ❷. Then you create a `RenderWindow` object ❸, which initializes all the other objects needed for the program, and start the rendering with a call to the `RenderWindow` object's `run()` method ❹.

The RenderWindow Constructor

The constructor on the `RenderWindow` class starts with the standard GLFW OpenGL setup you saw in [Chapter 9](#), including setting the window size, calling the render methods, and handling window and keyboard events. Then the constructor goes on with the following Game of Life-specific initializations:

```
class RenderWindow:
```

```
    def __init__(self, glider):
```

```
        -- snip --
```

```
        # create 3D
```

```
NX = 64
```

```
NY = 64
```

```
R = 4.0
```

```
r = 1.0
```

```
❶ self.torus = Torus(R, r, NX, NY)
```

```
❷ self.gol = GOL(NX, NY, glider)
```

```
# create a camera
```

```
❸ self.camera = OrbitCamera(5.0, 10.0)
```

```
# exit flag
```

```
❹ self.exitNow = False
```

```
# rotation flag
```

```
❺ self.rotate = True
```

```
# skip count
```

```
❻ self.skip = 0
```

First you set some parameters for the simulation, including the number of cells in the grid and the inner and outer radiuses of the torus. Then you create the `Torus` object using these parameters ❶, as well as the `GOL` object that will manage the simulation ❷. You also create the orbiting camera at a radius of 5 units from the origin and a height of 10 units from the XY plane ❸.

Next, you set the exit flag used to quit the program ❹, and you initialize the rotation flag to `True` ❺. Finally, you set a `skip` variable ❻,

which you'll use to control the frequency at which the simulation updates. You'll see how the `skip` variable works later in this section.

The `run()` and `step()` Methods

The `run()` method of the `RenderWindow` object is responsible for running the simulation, with help from the `step()` method. Let's take a look at the `run()` method first:

```
def run(self):

    # initializer timer

    glfw.glfwSetTime(0)

    t = 0.0

    ❶ while not glfw.glfwWindowShouldClose(self.win) and not
self.exitNow:

        # update every x seconds

        currT = glfw.glfwGetTime()

        ❷ if currT - t > 0.05:

            # update time

            t = currT

            # set viewport

            ❸ self.width, self.height = glfw.glfwGetFramebufferSize(self.win)

            self.aspect = self.width/float(self.height)

            glViewport(0, 0, self.width, self.height)
```

```
# clear

glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)

# build projection matrix

pMatrix = glutils.perspective(60.0, self.aspect, 0.1, 100.0)

mvMatrix = glutils.lookAt(self.camera.eye, self.camera.center,

                           self.camera.up)

# render

❷ self.torus.render(pMatrix, mvMatrix)

# step

❸ if self.rotate:

    self.step()

    glfw.glfwSwapBuffers(self.win)

# poll for and process events

    glfw.glfwPollEvents()

# end

    glfw.glfwTerminate()
```

The rendering scheme is designed to keep rendering frames in a loop until the window is closed or the ESC key is pressed ❶. Before you proceed, you check whether the time elapsed since the last render is greater than 0.05 seconds ❷. This helps maintain a maximum frame rate. Starting at ❸, you perform some standard OpenGL operations,

such as setting the viewport, clearing the screen, and computing the current transformation that needs to be set into the vertex shader. Then you render the torus ❹ and call the `step()` method ❺, which will rotate the camera and update the Game of Life simulation by one time step. Once the rendering is done, you swap the OpenGL buffers and poll for further window events. If you exit the loop, you call the `glfwTerminate()` method for cleanup.

Here's the `step()` method that increments the camera and the simulation:

```
def step(self):
```

```
    ❶ if self.skip == 9:

        # update GOL

        ❷ self.gol.update()

        ❸ colors = self.gol.get_colors()

        ❹ self.torus.set_colors(colors)

        # step

        ❺ self.torus.step()

        # reset

        ❻ self.skip = 0

        # update skip

        ❼ self.skip += 1

        # rotate camera
```

⑧ `self.camera.rotate()`

Every time this method is called, it rotates the camera by one step ⑧. You also want to update the Game of Life simulation, but doing so at the same rate that the camera moves would not be visually pleasing. You therefore use the `skip` variable to slow down the simulation by a factor of 9 relative to the camera motion. This variable starts from 0 and is incremented each time the `step()` method is called ⑦. When `skip` gets to 9 ①, you update the simulation by one time step. To do this, you first call the `GOL` class's `update()` method ②, which turns cells on or off according to the Conway's Game of Life rules. Then you get the updated cell colors from the simulation ③, set them to the torus ④, and call `torus.step()` ⑤, which will update the attribute buffers with the new colors. Finally, you reset the `skip` variable to 0 so the process can repeat ⑥.

Running the 3D Game of Life Simulation

Now we're ready to run the code. Enter the following at the terminal:

```
$ python gol_torus.py
```

Figure 10-8 shows the output.



Figure 10-8: A rendering of the Game of Life on a torus

The program will open a window that shows the torus you meticulously constructed, with the Game of Life simulation running on its surface! As the simulation evolves, try to find some of the familiar Game of Life patterns you saw in [Chapter 3](#). Notice that the light direction remains constant while the camera orbits the torus. As the camera turns, you'll be able to see the light and dark portions of the torus.

Now let's try the glider option:

```
$ python gol_torus.py --glider
```

[Figure 10-9](#) shows the output.



Figure 10-9: A Game of Life glider on a torus

Sit back and enjoy watching the lone glider make its way along the surface of the torus!

Summary

In this chapter, you implemented Conway's Game of Life on a torus. You learned how to compute the vertices of a torus and how to render it using OpenGL, and you saw how code can be adapted from one context (a flat rendering of the Game of Life simulation) to another (a 3D rendering of the same simulation). In the process, I hope you've gotten a more intuitive feeling for how the toroidal boundary conditions we discussed in [Chapter 3](#) work.

Experiments!

Here are a few experiments you can try with this project:

1. In this chapter's implementation, the torus is lit by a single light source. Try adding another light source in the shader code. Now the computed color of a vertex will be the sum of contributions from both light sources. Try changing the positions and colors of the light sources and see the effect on the torus lighting.
2. To get a representative view of the simulation, you defined a camera that revolves around the z-axis of the torus, in a plane parallel to the XY plane. Now create a camera that flies over the torus instead. Your camera will start by looking down at the torus along the negative z-axis and will move in a circle along the XZ plane, at a fixed distance from the center of the torus. Think about how you will compute the eye point, view direction, and up vector for each step of the movement.

The Complete Torus Rendering Code

Here's the complete listing for the file *torus.py*:

```
"""
```

```
torus.py
```

```
A Python OpenGL program that generates a torus.
```

```
Author: Mahesh Venkitachalam
```

```
"""
```

```
import OpenGL
```

```
from OpenGL.GL import *
```

```
import numpy as np
```

```
import math, sys, os
```

```
import glutils

import glfw

strVS = ""

# version 330 core

layout(location = 0) in vec3 aVert;

layout(location = 1) in vec3 aColor;

layout(location = 2) in vec3 aNormal;

uniform mat4 uMVMatrix;

uniform mat4 uPMatrix;

flat out vec3 vColor;

out vec3 vNormal;

out vec3 fragPos;

void main() {

    // transform vertex

    gl_Position = uPMatrix * uMVMatrix * vec4(aVert, 1.0);

    fragPos = aVert;

    vColor = aColor;

    vNormal = aNormal;

}
```

```
"""

strFS = """

# version 330 core

flat in vec3 vColor;

in vec3 vNormal;

in vec3 fragPos;

out vec4 fragColor;

void main() {

    vec3 lightPos = vec3(10.0, 10.0, 10.0);

    vec3 lightColor = vec3(1.0, 1.0, 1.0);

    vec3 lightDir = normalize(lightPos - fragPos);

    float diff = max(dot(vNormal, lightDir), 0.0);

    vec3 diffuse = diff * lightColor;

    float ambient = 0.1;

    vec3 result = (ambient + diffuse) * vColor.xyz;

    fragColor = vec4(result, 1.0);

}

"""

class Torus:
```

```
""" OpenGL 3D scene class """

# initialization

def __init__(self, R, r, NX, NY):

    global strVS, strFS

    # create shader

    self.program = glutils.loadShaders(strVS, strFS)

    glProvokingVertex(GL_FIRST_VERTEX_CONVENTION)

    self.pMatrixUniform = glGetUniformLocation(self.program,

                                                b'uPMatrix')

    self.mvMatrixUniform = glGetUniformLocation(self.program,

                                                b'uMVMatrix')

    # torus geometry

    self.R = R

    self.r = r

    # grid size

    self.NX = NX

    self.NY = NY

    # no. of points

    self.N = self.NX
```

```
self.M = self.NY

# time

self.t = 0

# compute parameters for glMultiDrawArrays

M1 = 2*self.M + 2

self.first_indices = [2*M1*i for i in range(self.N)]

self.counts = [2*M1 for i in range(self.N)]

# colors: {(i, j) : (r, g, b)}

# with NX * NY entries

self.colors_dict = self.init_colors(self.NX, self.NY)

# create an empty array to hold colors

self.colors = np.zeros((3*self.N*(2*self.M + 2), ), np.float32)

# get vertices, normals, indices

vertices, normals = self.compute_vertices()

self.compute_colors()

# set up vertex buffer objects

self.setup_vao(vertices, normals, self.colors)

def init_colors(self, NX, NY):

    """initialize color dictionary"""
```

```
colors = {}

c1 = [1.0, 1.0, 1.0]

for i in range(NX):

    for j in range (NY):

        colors[(i, j)] = c1

return colors

def compute_rt(self, R, alpha):

    # compute position of ring

    Tx = R*math.cos(alpha)

    Ty = R*math.sin(alpha)

    Tz = 0.0

    # rotation matrix

    RM = np.array([

        [math.cos(alpha), -math.sin(alpha), 0.0, 0.0],

        [math.sin(alpha), math.cos(alpha), 0.0, 0.0],

        [0.0, 0.0, 1.0, 0.0],

        [0.0, 0.0, 0.0, 1.0]

    ], dtype=np.float32)

    # translation matrix
```

```
TM = np.array([

    [1.0, 0.0, 0.0, Tx],

    [0.0, 1.0, 0.0, Ty],

    [0.0, 0.0, 1.0, Tz],

    [0.0, 0.0, 0.0, 1.0]

], dtype=np.float32)

return (RM, TM)

def compute_pt(self, r, theta, RM, TM):

    # compute point coords

    P = np.array([r*math.cos(theta), 0.0, r*math.sin(theta), 1.0],

                  dtype=np.float32)

    # print(P)

    # apply rotation - this also gives us the vertex normals

    NV = np.dot(RM, P)

    # normalize

    # NV = NV / np.linalg.norm(NV)

    # apply translation

    Pt = np.dot(TM, NV)

    return (Pt, NV)
```

```
def compute_vertices(self):

    """compute vertices for the torus

    returns np float32 array of n coords (x, y, z): shape (3*n, )

    """

    R, r, N, M = self.R, self.r, self.N, self.M

    # create an empty array to hold vertices/normals

    vertices = []

    normals = []

    # the points on the ring are generated on the X-Z plane

    # then they are rotated and translated into the correct

    # position on the torus

    # for all N rings around the torus

    for i in range(N):

        # for all M points around a ring

        for j in range(M+1):

            # compute angle theta of point

            theta = (j % M) * 2 * math.pi / M

            #---ring #1-----

            # compute angle
```

```
alpha1 = i*2*math.pi/N

# compute transforms

RM1, TM1 = self.compute_rt(R, alpha1)

# compute points

Pt1, NV1 = self.compute_pt(r, theta, RM1, TM1)

#---ring #2-----

# index of next ring

ip1 = (i + 1) % N

# compute angle

alpha2 = ip1*2*math.pi/N

# compute transforms

RM2, TM2 = self.compute_rt(R, alpha2)

# compute points

Pt2, NV2 = self.compute_pt(r, theta, RM2, TM2)

# store vertices/normals in right order for
GL_TRIANGLE_STRIP

vertices.append(Pt1[0:3])

vertices.append(Pt2[0:3])

# add normals
```

```
normals.append(NV1[0:3])

normals.append(NV2[0:3])

# return vertices and colors in correct format

vertices = np.array(vertices, np.float32).reshape(-1)

normals = np.array(normals, np.float32).reshape(-1)

# print(vertices.shape)

return vertices, normals

def compute_colors(self):

    """compute vertices for the torus

    returns np float32 array of n coords (x, y, z): shape (3*n, )

    """

    R, r, N, M = self.R, self.r, self.N, self.M

    # the points on the ring are generated on the X-Z plane

    # then they are rotated and translated into the correct

    # position on the torus

    # for all N rings around the torus

    for i in range(N):

        # for all M points around a ring

        for j in range(M+1):
```

```
# j value

jj = j % M

# store colors - same color applies to (V_i_j, V_ip1_j)

col = self.colors_dict[(i, jj)]

# get index into array

index = 3*(2*i*(M+1) + 2*j)

# set color

self.colors[index:index+3] = col

self.colors[index+3:index+6] = col

def setup_vao(self, vertices, normals, colors):

    # set up vertex array object (VAO)

    self.vao = glGenVertexArrays(1)

    glBindVertexArray(self.vao)

    # -----

    # vertices

    # -----

    self.vertexBuffer = glGenBuffers(1)

    glBindBuffer(GL_ARRAY_BUFFER, self.vertexBuffer)

    # set buffer data
```

```
glBufferData(GL_ARRAY_BUFFER, 4*len(vertices), vertices,

            GL_STATIC_DRAW)

# enable vertex attribute array

glEnableVertexAttribArray(0)

# set buffer data pointer

glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 0, None)

# normals

# -----

self.normalBuffer = glGenBuffers(1)

glBindBuffer(GL_ARRAY_BUFFER, self.normalBuffer)

# set buffer data

glBufferData(GL_ARRAY_BUFFER, 4*len(normals), normals,

            GL_STATIC_DRAW)

# enable vertex attribute array

glEnableVertexAttribArray(2)

# set buffer data pointer

glVertexAttribPointer(2, 3, GL_FLOAT, GL_FALSE, 0, None)

# -----

# colors
```

```
# -----

self.colorBuffer = glGenBuffers(1)

glBindBuffer(GL_ARRAY_BUFFER, self.colorBuffer)

# set buffer data

glBufferData(GL_ARRAY_BUFFER, 4*len(colors), colors,

             GL_STATIC_DRAW)

# enable color attribute array

glEnableVertexAttribArray(1)

# set buffer data pointer

glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 0, None)

# unbind VAO

glBindVertexArray(0)

def set_colors(self, colors):

    self.colors_dict = colors

    self.recalc_colors()

def recalc_colors(self):

    # get colors

    self.compute_colors()

    # bind VAO
```

```
glBindVertexArray(self.vao)

# -----

# colors

# -----

glBindBuffer(GL_ARRAY_BUFFER, self.colorBuffer)

# set buffer data

glBufferSubData(GL_ARRAY_BUFFER, 0, 4*len(self.colors),
self.colors)

# unbind VAO

glBindVertexArray(0)

# step

def step(self):

    # recompute colors

    self.recalc_colors()

# render

def render(self, pMatrix, mvMatrix):

    # use shader

    glUseProgram(self.program)

    # set proj matrix
```

```
glUniformMatrix4fv(self.pMatrixUniform, 1, GL_FALSE, pMatrix)

# set modelview matrix

glUniformMatrix4fv(self.mvMatrixUniform, 1, GL_FALSE,
mvMatrix)

# bind VAO

glBindVertexArray(self.vao)

# draw

glMultiDrawArrays(GL_TRIANGLE_STRIP, self.first_indices,

                  self.counts, self.N)

# unbind VAO

glBindVertexArray(0)
```

The Complete Game of Life Simulation Code

Here's the full code listing for the file *gol.py*.

```
"""
```

```
gol.py
```

```
Implements Conway's Game of Life.
```

```
Author: Mahesh Venkitachalam
```

```
"""
```

```
import numpy as np
```

```
class GOL:
```

```
    """GOL - class that implements Conway's Game of Life
```

```
    """
```

```
    def __init__(self, NX, NY, glider):
```

```
        """GOL constructor"""
```

```
        # a grid of NX x NY random values
```

```
        self.NX, self.NY = NX, NY
```

```
        if glider:
```

```
            self.addGlider(1, 1, NX, NY)
```

```
        else:
```

```
            self.grid = np.random.choice([1, 0], NX * NY, p=[0.2,
0.8]).reshape(NX, NY)
```

```
    def addGlider(self, i, j, NX, NY):
```

```
        """adds a glider with top left cell at (i, j)"""
```

```
        self.grid = np.zeros(NX * NY).reshape(NX, NY)
```

```
        glider = np.array([[0, 0, 1],
```

```
                            [1, 0, 1],
```

```
                            [0, 1, 1]])
```

```
        self.grid[i:i+3, j:j+3] = glider
```

```
def update(self):

    """update the GOL simulation by one time step"""

    # copy grid since we require 8 neighbors for calculation

    # and we go line by line

    newGrid = self.grid.copy()

    NX, NY = self.NX, self.NY

    for i in range(NX):

        for j in range(NY):

            # compute 8-neighbor sum

            # using toroidal boundary conditions - x and y wrap around

            # so that the simulation takes place on a toroidal surface

            total = (self.grid[i, (j-1) % NY] + self.grid[i, (j+1) % NY] +

                    self.grid[(i-1) % NX, j] + self.grid[(i+1) % NX, j] +

                    self.grid[(i-1) % NX, (j-1) % NY] + self.grid[(i-1) % NX, (j+1)

% NY] +

                    self.grid[(i+1) % NX, (j-1) % NY] + self.grid[(i+1) % NX,

(j+1) % NY])

            # apply Conway's rules

            if self.grid[i, j] == 1:

                if (total < 2) or (total > 3):
```

```
        newGrid[i, j] = 0

    else:

        if total == 3:

            newGrid[i, j] = 1

    # update data

    self.grid[:] = newGrid[:]

def get_colors(self):

    """returns a dictionary of colors"""

    colors = {}

    c1 = np.array([1.0, 1.0, 1.0], np.float32)

    c2 = np.array([0.0, 0.0, 0.0], np.float32)

    for i in range(self.NX):

        for j in range (self.NY):

            if self.grid[i, j] == 1:

                colors[(i, j)] = c2

            else :

                colors[(i, j)] = c1

    return colors
```

The Complete Camera Code

Here's the complete code in the file *camera.py*:

```
"""
```

```
camera.py
```

```
A simple camera class for OpenGL rendering.
```

```
Author: Mahesh Venkitachalam
```

```
"""
```

```
import numpy as np
```

```
import math
```

```
class OrbitCamera:
```

```
    """helper class for viewing"""
```

```
    def __init__(self, height, radius, beta_step=1):
```

```
        self.radius = radius
```

```
        self.beta = 0
```

```
        self.beta_step = beta_step
```

```
        self.height = height
```

```
        # initial eye vector is (-R, 0, -H)
```

```
        rr = radius/math.sqrt(2.0)
```

```
        self.eye = np.array([rr, rr, height], np.float32)
```

```
# compute up vector

self.up = self.__compute_up_vector(self.eye )

# center is origin

self.center = np.array([0, 0, 0], np.float32)

def __compute_up_vector(self, E):

    """compute up vector

    
$$N = (E \times k) \times E$$


    """

    #  $N = (E \times k) \times E$ 

    Z = np.array([0, 0, 1], np.float32)

    U = np.cross(np.cross(E, Z), E)

    # normalize

    U = U / np.linalg.norm(U)

    return U

def rotate(self):

    """rotate by one step and compute new camera parameters"""

    self.beta = (self.beta + self.beta_step) % 360

    # recalculate eye E

    self.eye = np.array([self.radius*math.cos(math.radians(self.beta)),
```

```
self.radius*math.sin(math.radians(self.beta)),  
  
self.height], np.float32)
```

```
# up vector
```

```
self.up = self.__compute_up_vector(self.eye)
```

The Complete RenderWindow Code

The complete code listing for *gol_torus.py*, including the `RenderWindow` class and the `main()` function, follows.

```
"""
```

```
gol_torus.py
```

```
Python OpenGL program that displays a torus.
```

```
Author: Mahesh Venkitachalam
```

```
"""
```

```
import OpenGL
```

```
from OpenGL.GL import *
```

```
import numpy, math, sys, os
```

```
import argparse
```

```
import glutils
```

```
import glfw
```

```
from torus import Torus
```

```
from camera import OrbitCamera

from gol import GOL

class RenderWindow:

    """GLFW Rendering window class"""

    def __init__(self, glider):

        # save current working directory

        cwd = os.getcwd()

        # initialize glfw - this changes cwd

        glfw.glfwInit()

        # restore cwd

        os.chdir(cwd)

        # version hints

        glfw.glfwWindowHint(glfw.GLFW_CONTEXT_VERSION_MAJOR, 3)

        glfw.glfwWindowHint(glfw.GLFW_CONTEXT_VERSION_MINOR, 3)

        glfw.glfwWindowHint(glfw.GLFW_OPENGL_FORWARD_COMPAT,
GL_TRUE)

        glfw.glfwWindowHint(glfw.GLFW_OPENGL_PROFILE,
glfw.GLFW_OPENGL_CORE_PROFILE)

        # make a window

        self.width, self.height = 640, 480
```

```
self.aspect = self.width/float(self.height)

self.win = glfw.glfwCreateWindow(self.width, self.height,
b'gol_torus')

# make context current

glfw.glfwMakeContextCurrent(self.win)

# initialize GL

glViewport(0, 0, self.width, self.height)

glEnable(GL_DEPTH_TEST)

#glClearColor(0.2, 0.2, 0.2, 1.0)

glClearColor(0.11764706, 0.11764706, 0.11764706, 1.0)

# set window callbacks

glfw.glfwSetMouseButtonCallback(self.win, self.onMouseButton)

glfw.glfwSetKeyCallback(self.win, self.onKeyboard)

# create 3D

NX = 64

NY = 64

R = 4.0

r = 1.0

self.torus = Torus(R, r, NX, NY)
```

```
self.gol = GOL(NX, NY, glider)

# create a camera

self.camera = OrbitCamera(5.0, 10.0)

# exit flag

self.exitNow = False

# rotation flag

self.rotate = True

# skip count

self.skip = 0

def onMouseButton(self, win, button, action, mods):

    # print 'mouse button: ', win, button, action, mods

    pass

def onKeyboard(self, win, key, scancode, action, mods):

    # print 'keyboard: ', win, key, scancode, action, mods

    if action == glfw.GLFW_PRESS:

        # ESC to quit

        if key == glfw.GLFW_KEY_ESCAPE:

            self.exitNow = True

        elif key == glfw.GLFW_KEY_R:
```

```
self.rotate = not self.rotate

def run(self):

    # initializer timer

    glfw.glfwSetTime(0)

    t = 0.0

    while not glfw.glfwWindowShouldClose(self.win) and not
self.exitNow:

        # update every x seconds

        currT = glfw.glfwGetTime()

        if currT - t > 0.05:

            # update time

            t = currT

            # set viewport

            self.width, self.height =
glfw.glfwGetFramebufferSize(self.win)

            self.aspect = self.width/float(self.height)

            glViewport(0, 0, self.width, self.height)

            # clear

            glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)

            # build projection matrix
```

```
pMatrix = glutils.perspective(60.0, self.aspect, 0.1, 100.0)

mvMatrix = glutils.lookAt(self.camera.eye, self.camera.center,
self.camera.up)

# render

self.torus.render(pMatrix, mvMatrix)

# step

if self.rotate:

    self.step()

glfw.glfwSwapBuffers(self.win)

# poll for and process events

glfw.glfwPollEvents()

# end

glfw.glfwTerminate()

def step(self):

    if self.skip == 9:

        # update GOL

        self.gol.update()

        colors = self.gol.get_colors()

        self.torus.set_colors(colors)
```

```
# step

self.torus.step()

# reset

self.skip = 0

# update skip

self.skip += 1

# rotate camera

self.camera.rotate()

# main() function

def main():

    print("Starting GOL. Press ESC to quit.")

    # parse arguments

    parser = argparse.ArgumentParser(description="Runs Conway's
    Game of Life simulation

                                on a Torus.")

    # add arguments

    parser.add_argument('--glider', action='store_true', required=False)

    args = parser.parse_args()

    # set args
```

```
glider = False

if args.glider:

    glider = True

rw = RenderWindow(glider)

rw.run()

# call main

if __name__ == '__main__':

    main()
```
